Codestash — Roadmap to Success

Status: expanded 2026-09-03 on `claude-redesign`, merged to `main` and deployed same day. Moved into `md-docs/roadmap/` alongside its own phase files 2026-09-04 — previously sat one level up on its own, the one file under `md-docs/` that wasn't a sibling of the folder holding its detail. This is the index — current state, the principles behind the sequencing, and a phase-by-phase map. Each phase's full detail (locked decisions, checklist, deferred items, exit condition) lives in its own sibling file in this same folder, kept short and searchable rather than one long document. Read alongside `../STORY.md` (how we got here) and `../ROLES-AND-BILLING-PLAN.md` (the org/role/plan design this builds toward).

This is written as if Codestash is going to be a real, paying-customer product — not a personal tool that happens to have a database attached.

Note on scope: only `roadmap/ROADMAP.md` (this index) is auto-loaded into every session via `CLAUDE.md`'s `@import`. The six other phase files in this same folder are read on demand when actually working on that phase — kept out of the always-loaded context on purpose, so a session touching Phase 4 doesn't have to load Phase 2's billing detail it doesn't need.

The honest current state

Not the optimistic version. Last updated 2026-09-03:

- A real DB/auth layer — sign-up, sign-in, workspaces, invites, all working end-to-end. **The catalog itself is 100% DB-backed now, with zero static content left anywhere** (`lib/data/` holds only shared types) — fixed 2026-09-04: the home page (`app/(main)/page.tsx`) was still rendering a hardcoded `CATEGORY_LIST`, showing every visitor the same 5 built-in category cards regardless of their actual org, an oversight from the original DB migration. Also reversed the same day: the catalog is no longer public — every route requires a real session (`proxy.ts` + a per-page server-verified check), a signed-out visitor is redirected to `/sign-in`. See `ROLES-AND-BILLING-PLAN.md` #5.
- **Exactly one real organization exists**, created by a direct database insert — no repeatable path exists yet for a second one.
- **Production was live but broken, now fixed** — `/` and every `/api/auth/*` call were 500ing because `NEXT_PUBLIC_APP_URL` in Vercel's Production env vars was missing its `https://` scheme. Corrected and confirmed returning 200, 2026-09-03.
- **CI now runs on every push** (`.github/workflows/ci.yml` — lint, `tsc --noEmit`, build) — still zero automated test suite; that's Phase 6, not this. CI's build step needs `DATABASE_URL`, `BETTER_AUTH_SECRET`, and `NEXT_PUBLIC_APP_URL` added as GitHub Actions repo secrets before it can actually pass — not yet confirmed done.
- **Zero billing** — nothing charges anyone anything today.
- The roles/plan model's numbers are locked now (Plan A/B/C's seat pricing, `lib/config/plan-limits.ts`) and the 3-role structure is confirmed (Organization Manager = `owner`, mapped to `admin` / `member`). **Permission checks are real now too** — `requireOrgRole` gates every org-scoped write. What's still not true: `plan-limits.ts`'s actual limits (max categories, custom backgrounds) aren't wired into any real feature gate

yet, and seat-cap enforcement is deliberately deferred to Phase 2 (no `seatsPurchased` field exists to enforce against).

- **Create category UI exists** (Phase 1). **Create/edit UI for manuals and snippets exists too, as of 2026-09-04** — owner/admin only, same `requireOrgRole` pattern. One real limitation: it only supports flat, single-block sections (no nesting, no multi-block sections), so editing richer existing content (this roadmap manual, `mastering-git`, etc.) through it isn't offered at all — the Edit button only appears when a manual's actual structure is flat-compatible (`lib/helpers/manual-edit-compat.ts`).

Principles — corrections, not just a continuation

Named specifically so they don't repeat under a new coat of paint. Each phase's locked decisions trace back to one of these.

1. **Stop creating state by hand.** The org, its categories, and (until recently) its owner membership were all created by direct inserts or one-off scripts — that exact pattern broke sidebar drag-and-drop. Nothing gets created except through the app's real flow, including in development.
2. **Build the permission/plan-limit layer before more features.** Every feature shipped so far went in with no thought to who's allowed to do it — backwards for a product whose business model is tiered, paid plans.
3. **Billing earlier than feels comfortable.** "Paying creates an organization" is the foundation the whole design sits on. An ugly, working checkout beats a beautiful app with no way to charge anyone.
4. **Fix what's live before designing what's next.** An unresolved production outage sat underneath this entire plan — Phase 0 exists because of it, and is now resolved.
5. **Bring in a safety net exactly when the stakes go up.** No tests, no CI is fine for a personal tool. It stops being fine once plans, seats, and permissions gate what real people can do and pay for.

The phases

#	Phase	Exit condition	Detail
0	**Stabilize** — nearly done	Production serves every route without a 500 (✓); broken builds can't merge silently (pending CI repo secrets)	00-stabilize.md
1	Foundations	No org/category/membership is ever created outside the app's own code paths	01-foundations.md
2	Billing	A second real, paying org can exist without a developer touching the database	02-billing.md
3	Core product completeness	A workspace's content is fully self-service	03-core-product.md
4	Team features	A member's capabilities are governed by their role, not just membership	04-team-features.md
5	Polish — original UX work	The sidebar UX work done right, with real architecture underneath	05-polish.md

#	Phase	Exit condition	Detail
6	Scale readiness	Ready to hold up under real, paying usage	06-scale-readiness.md

One decision in the phase details is still flagged as **confirm-before-building** rather than fully locked: the Stripe/Vitest choices (Phases 2 and 6). The Organization Manager → better-auth role mapping (Phase 1) was confirmed 2026-09-04. Everything else in each phase file is written to be built as-is, not re-debated.

Infra independence

None of the above depends on staying on Neon or Vercel. The org/role/plan model, the permission-check pattern, and the billing hook points are the same regardless of what's underneath. The one discipline required: keep database access behind `lib/db` and `lib/actions`, and auth behind `lib/auth` — so a future move (Neon → Appwrite, Vercel → Cloudflare) only touches those adapter layers, never the business logic on top.

Where this leaves us

Phase 0's checklist is done except confirming the CI repo secrets are added — the rest of that phase (the prod outage, CI itself) is fixed and live. The static-to-DB migration wasn't originally scoped as a Phase 0 item, but it's done too, ahead of its natural home in Phase 3. The sequencing for what's left is still deliberate — foundations before billing, billing before more product surface, product before team features, team features before polish, polish before hardening. Skipping ahead (more UI before the permission layer exists, for instance) is exactly the pattern this roadmap exists to correct.

Bringing this into the app itself

Done, not just planned — this file and its sibling phase files are browsable in-app as the "Roadmap to Success" manual under the `codestash` category, alongside "The Story" (`STORY.md`), "Project Setup & Context" (`SETUP.md`), "Organizations, Roles & Billing Plan" (`ROLES-AND-BILLING-PLAN.md`), and — new as of 2026-09-04 — "Project Rules" (`RULES.md`), which never had an in-app manual before.

Rewritten 2026-09-04: `scripts/update-roadmap-manual.ts` and `scripts/update-doc-family-manuals.ts` used to hand-duplicate each source file's content as literal strings inside the script — which drifted out of sync with the real `.md` files three separate times in one day before being rewritten. They now read the real files from disk and parse them (via `scripts/lib/markdown-to-manual-sections.ts`) into the manual's block format — there's exactly one copy of this content now, not two. Re-run either script (safe, idempotent) whenever the `.md` files it covers change; the old `seed-*.ts` *scripts stay in the repo only as a historical record, same principle as the category seed scripts in* `roadmap/01-foundations.md` *— the* `update-*.ts` scripts alone are sufficient to bootstrap a manual from nothing now, no separate seed step needed.

One known, disclosed simplification: `ContentBlock` has no table type, so a markdown table (e.g. this file's phase table) becomes a list of "Header: cell — Header: cell" rows instead of a real table. Single `*asterisk*` italics also aren't supported by the inline renderer (`parseInline` only handles `**bold**` and `` `code` ``) and show as literal asterisks — narrow, cosmetic, not worth a table block type or a richer inline parser for the one or two places it'd matter today.